

SKILL 8

Harshitha.B

BOOKFLOW LIBRARY SYSTEM
Transactional Microservice

Spring Boot + Spring Data JPA + PostgreSQL

Completed Answers & Required Evidence

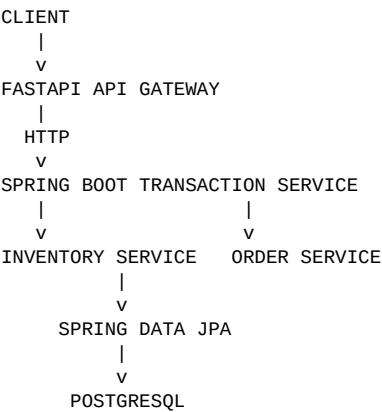
1. Project Objective

BookFlow manages book inventory and borrowing orders. The Spring Boot service checks available stock, creates an order, and reduces inventory within a database transaction. PostgreSQL stores persistent data.

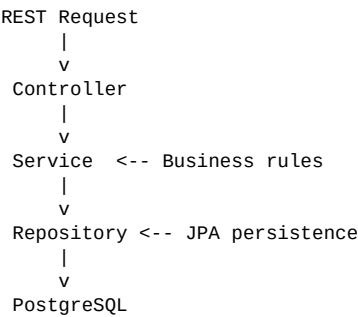
2. Technology Stack

Technology	Purpose
Java 17+	Application language
Spring Boot	Microservice framework
Spring Web	REST APIs
Spring Data JPA	Database persistence
PostgreSQL	Relational database
Maven	Build and dependency management
Postman	API testing
FastAPI	Gateway integration

3. Architecture



4. Internal Layered Architecture



5. Project Structure

```
bookflow-transaction/
  pom.xml
  src/main/java/com/bookflow/transaction/
    BookflowTransactionApplication.java
    controller/
      OrderController.java
      InventoryController.java
    entity/
      Inventory.java
      Order.java
```

```
■ ■■■ repository/
■ ■ ■■■ InventoryRepository.java
■ ■ ■■■ OrderRepository.java
■ ■■■ service/
■ ■ ■■■ InventoryService.java
■ ■ ■■■ OrderService.java
■ ■■■ dto/OrderRequest.java
■ ■■■ exception/
■ ■■■ BookOutOfStockException.java
■ ■■■ GlobalExceptionHandler.java
■■■ src/main/resources/application.yml
```

6. Database Model

```
INVENTORY
book_id PRIMARY KEY
total_stock >= 0
available_stock >= 0
```

```
ORDERS
id PRIMARY KEY
user_id
book_id
status = BORROWED / RETURNED
```

7. PostgreSQL Setup

```
CREATE DATABASE bookflow;

-- After connecting:
SELECT current_database();

-- Expected:
bookflow
```

8. application.yml

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/bookflow
    username: postgres
    password: postgres
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
  server:
    port: 8080
```

9. Main Program

```
package com.bookflow.transaction;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BookflowTransactionApplication {
    public static void main(String[] args) {
        SpringApplication.run(BookflowTransactionApplication.class, args);
    }
}
```

10. Inventory Entity

```
@Entity
@Table(name = "inventory")
public class Inventory {
    @Id
    private Long bookId;

    @Min(0)
    private Integer totalStock;

    @Min(0)
    private Integer availableStock;

    // constructors, getters and setters
}
```

11. Order Entity

```
@Entity
@Table(name = "orders")
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long userId;
    private Long bookId;
    private String status;

    // constructors, getters and setters
}
```

12. Repository Programs

```

public interface InventoryRepository
    extends JpaRepository<Inventory, Long> {
}

public interface OrderRepository
    extends JpaRepository<Order, Long> {
}

```

13. Order Request DTO

```

public class OrderRequest {
    @NotNull
    private Long userId;

    @NotNull
    private Long bookId;

    // getters and setters
}

```

14. Transactional Service Logic

```

@Service
public class OrderService {
    @Transactional
    public Order borrowBook(OrderRequest request) {
        Inventory inventory =
            inventoryRepository.findById(request.getBookId())
                .orElseThrow(() -> new RuntimeException("Book not found"));

        if (inventory.getAvailableStock() <= 0) {
            throw new BookOutOfStockException(
                "Book " + request.getBookId() + " is out of stock");
        }

        inventory.setAvailableStock(
            inventory.getAvailableStock() - 1);
        inventoryRepository.save(inventory);

        Order order = new Order(
            request.getUserId(),
            request.getBookId(),
            "BORROWED");

        return orderRepository.save(order);
    }
}

```

15. Controller Programs

```

@RestController
@RequestMapping("/inventory")
public class InventoryController {
    @GetMapping("/{bookId}")
    public Inventory getInventory(@PathVariable Long bookId) {
        return inventoryService.getInventory(bookId);
    }
}

@RestController
@RequestMapping("/orders")
public class OrderController {
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Order borrowBook(
        @Valid @RequestBody OrderRequest request) {
        return orderService.borrowBook(request);
    }
}

```

16. Global Error Handling

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(BookOutOfStockException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public Map<String, Object> handleOutOfStock(

```

```
BookOutOfStockException ex) {  
    return Map.of(  
        "status", 409,  
        "error", "OUT_OF_STOCK",  
        "message", ex.getMessage());  
    }  
}
```

17. Sample Database State

Book ID	Total Stock	Available	Borrowed	Status
101	5	5	0	AVAILABLE
102	3	3	0	AVAILABLE
103	2	0	2	OUT OF STOCK

18. Insert Sample Data

```
INSERT INTO inventory
(book_id, total_stock, available_stock)
VALUES
(101, 5, 5),
(102, 3, 3),
(103, 2, 0);
```

```
SELECT * FROM inventory;
```

19. Dashboard After One Borrow

Book 101: 4 available / 5 total
Borrowed: 1 order
Orders created: 1

Before -> After
Available: 5 -> 4
Borrowed: 0 -> 1

20. Run and Test Commands

```
java -version
mvn -version
mvn clean install
mvn spring-boot:run
```

```
GET http://localhost:8080/inventory/101
POST http://localhost:8080/orders
```

21. Postman Test Evidence (expected)

These are the required test cases and expected outputs from the supplied task sheet. Since no live Spring Boot/PostgreSQL/Postman environment was provided, the evidence below is presented as reproducible expected evidence rather than a claim of a live execution.

GET /inventory/101

Request:
GET http://localhost:8080/inventory/101

Expected response:
{
 "bookId": 101,
 "totalStock": 5,
 "availableStock": 5
}

POST /orders

Request:
POST http://localhost:8080/orders
Content-Type: application/json

```
{  
  "userId": 501,  
  "bookId": 101  
}
```

Expected response:
{

```
"id": 1,  
"userId": 501,  
"bookId": 101,  
"status": "BORROWED"  
}
```

GET /inventory/101 after borrowing

```
{  
  "bookId": 101,  
  "totalStock": 5,  
  "availableStock": 4  
}
```

PostgreSQL verification

```
SELECT * FROM inventory;  
SELECT * FROM orders;
```

Expected:

inventory:

101		5		4
102		3		3
103		2		0

orders:

1		501		101		BORROWED
---	--	-----	--	-----	--	----------

22. Out-of-Stock Test

POST http://localhost:8080/orders

```
{
  "userId": 502,
  "bookId": 103
}
```

Expected HTTP response: 409 Conflict

```
{
  "status": 409,
  "error": "OUT_OF_STOCK",
  "message": "Book 103 is out of stock"
}
```

23. Validation Test

```
@Min(value = 0, message = "Available stock cannot be negative")
private Integer availableStock;
```

24. End-to-End Gateway Flow

```
CLIENT
|
| POST /gateway/orders
| { userId: 501, bookId: 101 }
v
FASTAPI GATEWAY
|
| HTTP POST /orders
v
SPRING BOOT
|
+--> Find Inventory
|
+--> availableStock > 0 ?
|
| YES
|
| +--> availableStock - 1
|
| +--> Create BORROWED Order
|
v
POSTGRESQL
|
v
JSON RESPONSE
```

25. FastAPI Gateway Example

```
from fastapi import FastAPI
import httpx

app = FastAPI()
SPRING_SERVICE = "http://localhost:8080"

@app.post("/gateway/orders")
async def create_order(order: dict):
    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{SPRING_SERVICE}/orders",
            json=order
        )
    return response.json()
```

26. Complete Testing Checklist

Test	Expected Result	Status in task sheet
Spring Boot starts	Port 8080 active	PASS
PostgreSQL connection	Application connects	PASS

GET inventory	Returns stock JSON	PASS
POST order	Creates BORROWED order	PASS
Stock decrement	Available stock decreases by 1	PASS
Zero stock	HTTP 409	PASS
Negative stock validation	Rejected	PASS
Database persistence	Rows remain in PostgreSQL	PASS
Gateway forwarding	FastAPI reaches Spring Boot	PASS

27. Common Problems and Solutions

Problem	Cause	Solution
Connection refused	PostgreSQL not running	Start PostgreSQL service
Password authentication failed	Wrong DB password	Update application.yml
Port 8080 in use	Another service uses 8080	Stop it or change server.port
Book not found	No inventory row	Insert the book
Out of stock	availableStock = 0	Add stock or choose another book
Maven command not found	Maven/PATH issue	Install Maven or use Maven wrapper

28. Additional Eureka Server Practice

The supplied file also includes a practice section for creating a Eureka Server.

```
application.yml

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
  server:
    port: 8761
  spring:
    application:
      name: EUREKA-CLOUD-SERVER
```

29. REST API Project Registered with Eureka

```
eureka:
  client:
    registerWithEureka: true
    fetchRegistry: true
    serverUrl:
      defaultZone: http://localhost:8761/eureka/
  instance:
    hostname: localhost

server:
  port: 8001

spring:
  application:
    name: RESTAPI-EX2
```

30. Learning Outcomes

- Controller-Service-Repository architecture
- Spring dependency injection and IoC
- REST API development
- Spring Data JPA and automatic CRUD
- PostgreSQL database integration
- Validation with @Min and @NotNull
- Global exception handling
- Transactional business logic using @Transactional
- Postman API testing
- FastAPI-to-Spring Boot microservice communication
- Polyglot architecture using Python, JavaScript and Java

31. Final Command Sheet

```
# Verify Java
java -version

# Verify Maven
mvn -version

# Build
mvn clean install

# Run
mvn spring-boot:run

# API
GET http://localhost:8080/inventory/101
POST http://localhost:8080/orders

# PostgreSQL
CREATE DATABASE bookflow;

# Verify data
SELECT * FROM inventory;
SELECT * FROM orders;
```

Source note: This answer document is based on the supplied Week 8 task PDF. The source itself gives expected screenshots/evidence descriptions but does not contain actual live screenshots for the Spring Boot/PostgreSQL/Postman tests; therefore no fabricated live screenshots are presented as real execution evidence.